

Centro de Comercio y Servicios

Análisis y Desarrollo de Software

Instructor: Raúl Gómez

json-server

json-server es una librería de Node.js que permite generar un servidor API REST completo utilizando solo un archivo JSON como base de datos. Es ideal para proyectos en los que necesitas probar endpoints o simular un backend sin implementar uno real.

json-server

Inicializar un proyecto de Node.js

```
npm init -y
```

json-server

package.json

```
{  
  "name": "auth",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC"  
}
```

json-server

Instalar json-server

```
npm install json-server@0.17.4
```

json-server

db.json

```
{
  "users": [],
  "productos": [
    {
      "id": 1,
      "nombre": "Teclado",
      "precio": 150000
    },
    {
      "id": 2,
      "nombre": "Mouse",
      "precio": 80000
    }
  ]
}
```

json-server

server.js:

```
const jsonServer = require('json-server');
const cors = require('cors');

const app = jsonServer.create();
const router = jsonServer.router('db.json');

app.db = router.db;

app.use(cors());
app.use(router);
app.listen(3000, () => {
  console.log('Servidor corriendo en http://localhost:3000');
});
```

json-server

Iniciar el servidor

```
node server.js
```

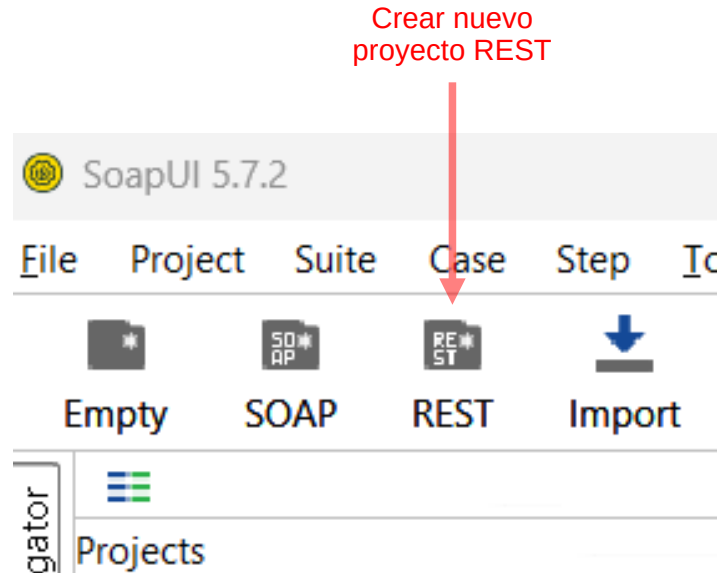

SoapUI

soapui.org



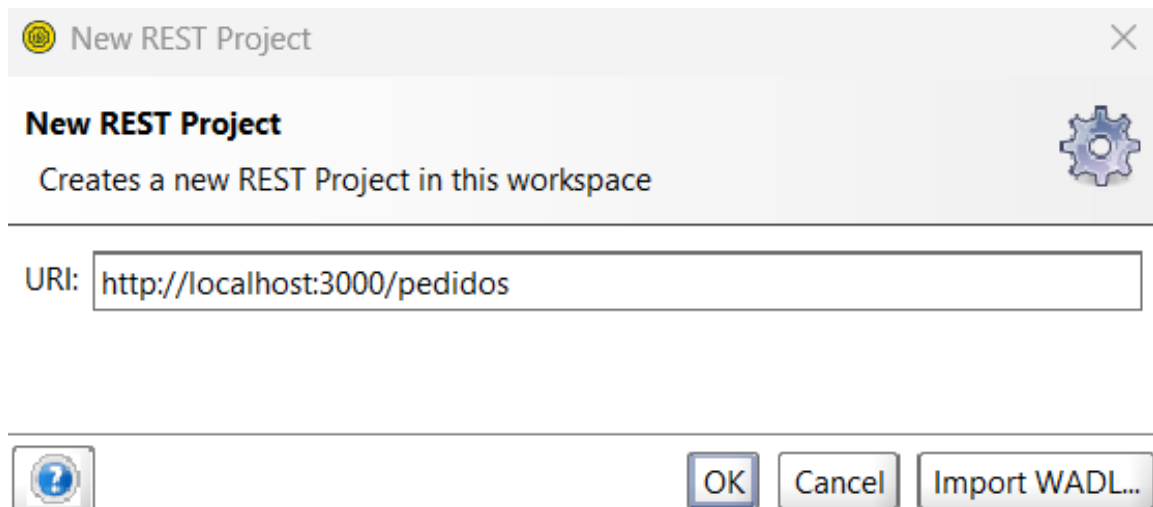
SoapUI

Crear proyecto REST



SoapUI

Configurar URI



SoapUI

Petición GET

Ejecutar
petición



The screenshot shows the SoapUI interface for a REST client. The top bar indicates 'Request 1' with a status of 'GET'. Below this, the 'Method' is set to 'GET', the 'Endpoint' is 'http://localhost:3000', and the 'Resource' is '/pedidos'. The 'Parameters' section is empty. The main area displays a table with columns 'Name', 'Value', 'Style', and 'Level', which is currently empty. To the right, the 'Raw' tab is selected, showing a JSON response. The JSON is a list of three objects, each representing a restaurant item with an 'id', 'platos' (plates), 'precio' (price), and 'idRestaurante' (restaurant ID).

```
1 [
2 {
3   "id": 1,
4   "platos": [
5     "bLrhfdw5te",
6     "SWRGGGRo55"
7   ],
8   "precio": "11",
9   "idRestaurante": "10"
10 },
11 {
12   "id": 2,
13   "platos": [
14     "ZKKLPmE5bB",
15     "yzE1WmLnIn"
16   ],
17   "precio": "74",
18   "idRestaurante": "16"
19 },
20 {
21   "id": 3,
22   "platos": [
23     "zc8az2QPhw",
24     "McPXB4mpG"
25   ],
26   "precio": "54",
27   "idRestaurante": "14"
28 }
```

json-server

Resultado:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- Endpoint:** http://localhost:3000
- Resource:** /productos

The response is displayed in the JSON tab, showing an array of two product objects:

```
1 [
2   {
3     "id": 1,
4     "nombre": "Teclado",
5     "precio": 150000
6   },
7   {
8     "id": 2,
9     "nombre": "Mouse",
10    "precio": 80000
11  }
12 ]
```

json-server

api.js:

```
import axios from "axios";

const API = axios.create({
  baseURL: "http://localhost:3000"
});

export const obtenerProductos = () => {
  return API.get("/productos");
};
```

json-server

ProductoLista.jsx:

```
import { useEffect, useState } from "react";
import { obtenerProductos } from "../services/api";

function ProductoLista() {

  const [productos, setProductos] = useState([]);

  useEffect(() => {
    cargarProductos();
  }, []);

  const cargarProductos = async () => {
    const res = await obtenerProductos();
    setProductos(res.data);
  };

  return (
    <div>
      <h2>Productos</h2>

      {productos.map(product => (
        <p key={product.id}>{product.nombre}</p>
      ))}

    </div>
  );
}
```

export default ProductoLista;

12/04/26

json-server

App.jsx:

```
import './App.css'
import ProductLista from './components/ProductLista'

function App() {
  return (
    <div>
      <ProductLista />
    </div>
  )
}

export default App
```


json-server

Ejecutar el proyecto

Productos

Teclado

Mouse

json-server-auth

json-server-auth es una extensión para json-server que agrega funcionalidad de autenticación y autorización. Incluye soporte para inicio de sesión, registro de usuarios y permisos basados en roles, utilizando JSON Web Tokens (JWT) para manejar la autenticación. También utiliza bcrypt para cifrar las contraseñas de forma segura.

json-server-auth

Instalar json-server-auth

```
npm install json-server-auth
```

json-server-auth

rules.json

```
{  
  "productos": 660  
}
```

json-server-auth

server.js:

```
const jsonServer = require('json-server');
const auth = require('json-server-auth');
const cors = require('cors');
const rules = require('./rules.json');

const app = jsonServer.create();
const router = jsonServer.router('db.json');

app.db = router.db;

app.use(cors());
app.use(auth.rewriter(rules));
app.use(auth);
app.use(router);
app.listen(3000, () => {
  console.log('Servidor corriendo en http://localhost:3000');
});
```

json-server-auth

Resultado:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- Endpoint:** http://localhost:3000
- Resource:** /productos

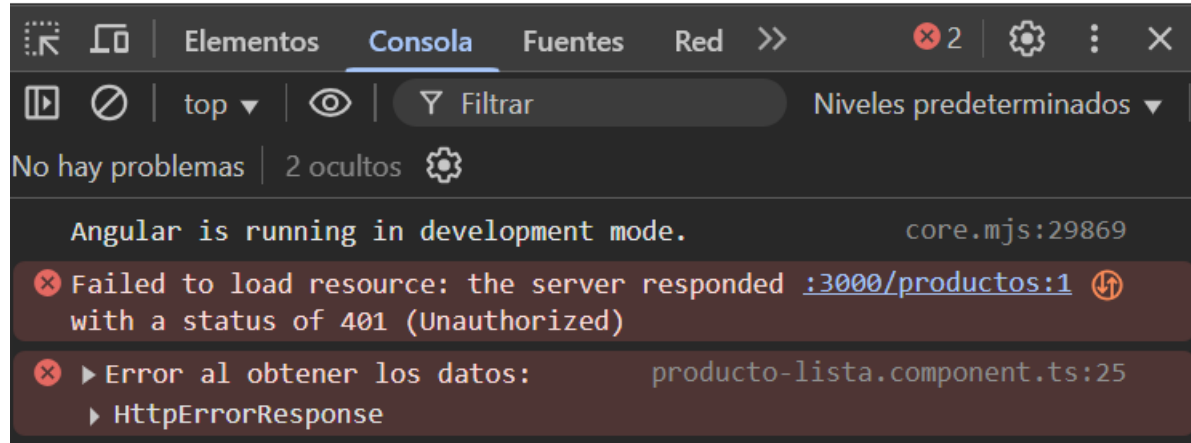
The response is displayed in the bottom right pane, showing a single entry with the message:

Name	Value	Style	Level
1	"Missing authorization header"		

json-server-auth

Resultado:

[]



json-server-auth

Resultado:

The screenshot shows a REST client interface with the following details:

- Method:** POST
- Endpoint:** http://localhost:3000
- Resource:** /register

The **Request** tab is active, displaying a table with headers: Name, Value, Style, and Level. Below the table, the **Raw** tab shows the request body as a JSON object:

```
{  "email": "admin@correo.com",  "password": "123456"}
```

The **Response** tab is also visible, showing the server's response as a JSON object:

```
{  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0:eyJ1e28iOiB1b3RlciIsImVtYWlsIjoiYWRhbmQ@correo.com",  "user": {    "email": "admin@correo.com",    "id": 1  }}
```


json-server-auth

db.json

```
{
  "users": [
    {
      "email": "admin@correo.com",
      "password": "$2a$10$7WgD.qmr0dUsq7sMXTTWQuLE6uZmyhw00cXDjInCV6Sq8NvMi3Yce",
      "id": 1
    }
  ],
  "productos": [
    {
      "id": 1,
      "nombre": "Teclado",
      "precio": 150000
    },
    {
      "id": 2,
      "nombre": "Mouse",
      "precio": 80000
    }
  ]
}
```

json-server-auth

Resultado:

The screenshot shows a REST client interface with the following details:

- Method:** POST
- Endpoint:** http://localhost:3000
- Resource:** /login
- Request Body (Raw):**

```
{  
  "email": "admin@correo.com",  
  "password": "123456"  
}
```
- Response Body (Raw):**

```
{  
  "accessToken": "eyJhbGciOiJIUzI1LCJ0eSI6InR5cGUzIn0",  
  "user": {  
    "email": "admin@correo.com",  
    "id": 1  
  }  
}
```

json-server-auth

Resultado:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- Endpoint:** http://localhost:3000
- Resource:** /productos

The **Request** tab is active, displaying a table of headers:

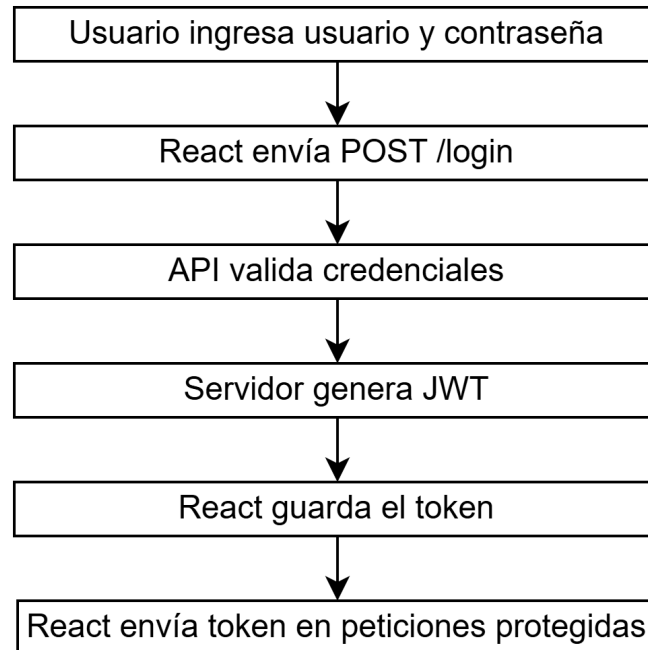
Name	Value	Style	Level
Authorization	Bearer eyJhbGciOi...	HEADER	RESOURCE

The **Response** tab is also visible, showing the JSON data:

```
[
  {
    "id": 1,
    "nombre": "Teclado",
    "precio": 150000
  },
  {
    "id": 2,
    "nombre": "Mouse",
    "precio": 80000
  }
]
```

Flujo de autenticación

El estándar utilizado es JSON Web Token.



json-server-auth

api.js:

```
import axios from "axios";

const API = axios.create({
  baseURL: "http://localhost:3000"
});

export const login = (data) => API.post("/login", data);

export const obtenerProductos = (token) => {
  return API.get("/productos", {
    headers: {
      Authorization: `Bearer ${token}`
    }
  });
};
```

json-server-auth

Login.jsx:

```
import { useState } from "react";
import { login } from "../services/api";

function Login({ setToken }) {

  const [email, setUsername] = useState("");
  const [password, setPassword] = useState("");

  const iniciarSesion = async () => {

    const res = await login({
      email,
      password
    });

    const token = res.data.accessToken;
    localStorage.setItem("token", token);
    setToken(token);

  };

  return (

    <div>

      <h2>Login</h2>

      <input
        placeholder="Correo"
        onChange={(e) => setUsername(e.target.value)}
      />

      <input
        type="password"
        placeholder="Password"
        onChange={(e) => setPassword(e.target.value)}
      />

      <button onClick={iniciarSesion}>
        Ingresar
      </button>

    </div>

  );

}
```

export default Login;

12/04/26

json-server-auth

ProductoLista.jsx:

```
import { useEffect, useState } from "react";
import { obtenerProductos } from "../services/api";

function ProductoLista({ token }) {

  const [productos, setProductos] = useState([]);

  useEffect(() => {
    cargarProductos();
  }, [token]);

  const cargarProductos = async () => {
    const res = await obtenerProductos(token);
    setProductos(res.data);
  };

  if (!productos) return <p>Cargando...</p>;

  return (
    <div>
      <h2>Productos</h2>

      {productos.map(product => (
        <p key={product.id}>{product.nombre}</p>
      ))}

    </div>
  );
}

export default ProductoLista;
```

12/04/26

json-server-auth

App.jsx:

```
import { useState } from "react";
import Login from "../components/Login";
import ProductoLista from "../components/ProductoLista";

function App() {
  const [token, setToken] = useState(
    localStorage.getItem("token")
  );

  return (
    <div>
      <h1>Autenticación JWT</h1>
      {!token ? (
        <Login setToken={setToken} />
      ) : (
        <ProductoLista token={token} />
      )}
    </div>
  );
}

export default App;
```


json-server-auth

Ejecutar el proyecto

Autenticación JWT

Login

Ingresar



Autenticación JWT

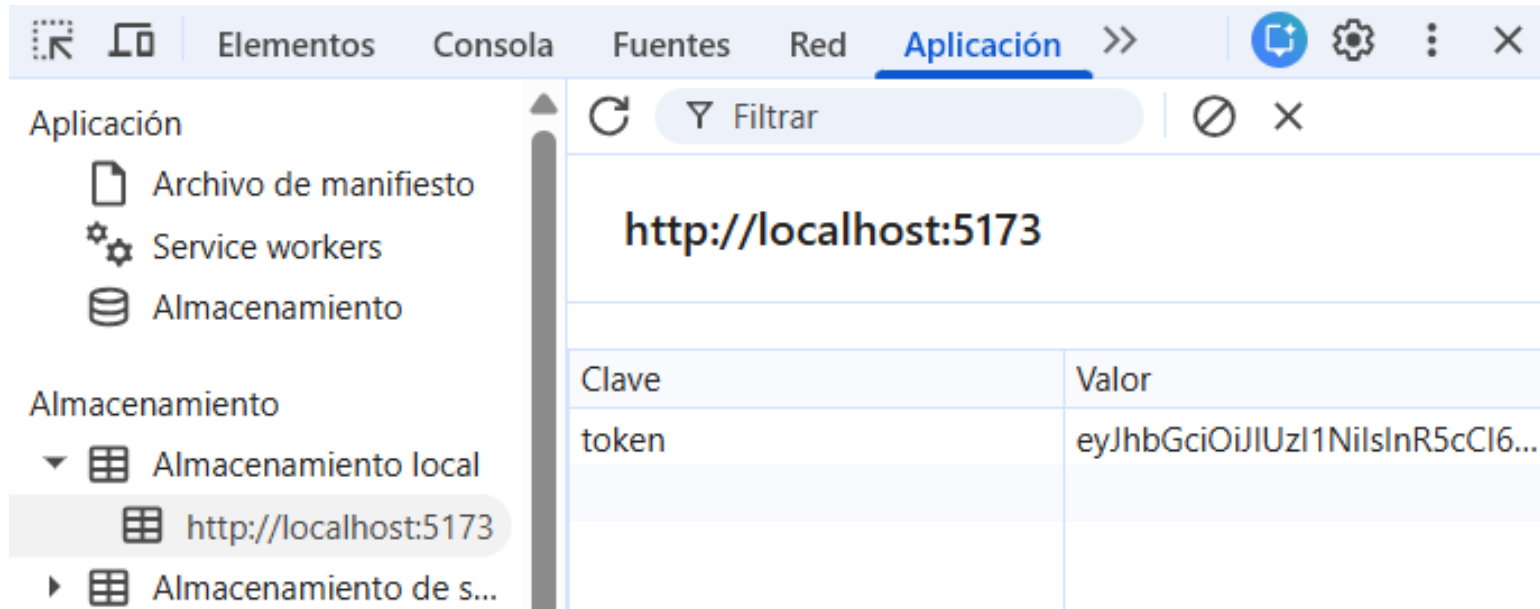
Productos

Teclado

Mouse

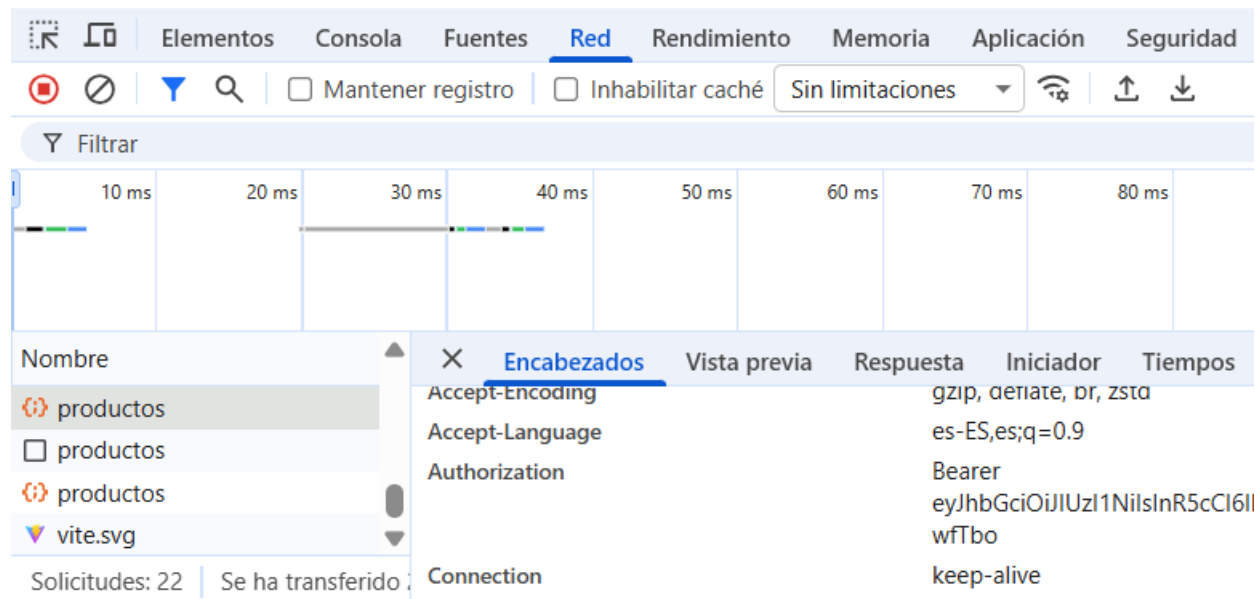
json-server-auth

Resultado:



json-server-auth

Resultado:



Ejercicios prácticos

Ejercicio 1

Condicionar botones o acciones (editar/eliminar) solo si el usuario tiene un rol adecuado.

Ejercicios prácticos

Ejercicio 2

Crear un nuevo usuario mediante POST /register

Tareas:

Crear RegisterComponent

Validar que el correo no exista

Crear usuario con { email, password, role }

Ejercicios prácticos

Ejercicio 3

Cerrar sesión automáticamente después de X minutos sin actividad.

Tareas:

Usar eventos de mousemove, keydown, etc.

Reiniciar contador con cada actividad

Si el temporizador llega a 0, cerrar sesión y redirigir.

Ejercicios prácticos

Ejercicio 4

Crear una vista para que el administrador pueda listar, editar o eliminar usuarios.

Tareas:

Solo accesible a rol admin.

Mostrar tabla de usuarios (/users)

Ejercicios prácticos

Ejercicio 5

Registrar un usuario y simular la activación con un enlace (usando un token simple o un parámetro en la URL).

Tareas:

Usuario se registra → POST /register

Se muestra mensaje: “Activa tu cuenta con el enlace”

Al hacer clic, se accede a /activar/:id

En ActivarComponent, se actualiza un campo activo: true en el usuario (requiere PATCH /users/:id)

Ejercicios prácticos

Ejercicio 6

Mostrar un dashboard que carga datos específicos del usuario autenticado.

Tareas:

Login → obtener user.id

GET /dashboard?userId=ID

Mostrar información (productos comprados, estadísticas, etc.)

Ejercicios prácticos

Ejercicio 7

Registrar y mostrar las sesiones iniciadas por el usuario.

En el login:

Hacer POST a /sesiones con userId y timestamp

En HistorialComponent, hacer GET /sesiones?userId=ID

Ejercicios prácticos

Ejercicio 8

Eliminar token y redirigir al login.

Tareas:

Borrar la cookie.

Redireccionar.

Mostrar mensaje de cierre de sesión.